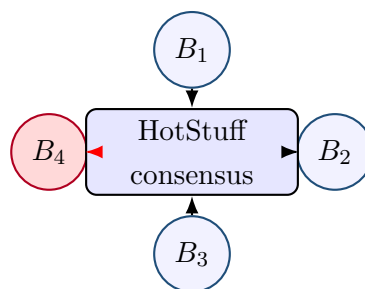


Skripta za učenje HotStuff algoritma

BFT konsenzus kroz bankarski case study

Ognjen Peric

June 22, 2026



Kako citati ovu skriptu

Ova skripta je napravljena iz govornickih beleški za prezentaciju, ali je prosirena u formu materijala za učenje. Ideja nije da se samo zapamte faze protokola, vec da se razume zasto HotStuff uvodi QC, lock, tri faze i Pacemaker. Kroz celu skriptu koristimo isti primer: permissioned bankarski sistem u kome vise banaka i regulator odrzavaju zajednicki ledger.

Contents

1 Motivacija: zasto nam treba konsenzus?	4
1.1 Zasto je redosled bitan?	4
2 Byzantine model: sta znaci da replika moze biti zlonamerna?	5
3 Model HotStuff-a	5
3.1 Zasto bas $n = 3f + 1$?	6
4 Bankarski case study	6
5 Osnovni pojmovi	7
5.1 Node	7
5.2 Branch	7
5.3 Vote	8
5.4 Quorum Certificate	8
6 Basic HotStuff: faze jednog view-a	8
6.1 New-view: gde je ledger stao?	9
6.2 Prepare faza: lider predlaze novi zahtev	9
6.3 Pre-commit faza: lider dokazuje da je prikupio prepare glasove	10
6.4 Commit faza: replike se zakljucavaju	10
6.5 Decide faza: zahtev se izvorsava	10
7 safeNode pravilo	11
7.1 Safety rule	11
7.2 Liveness rule	12
8 Safety: zasto dve konfliktne odluke ne mogu proci?	12
8.1 Quorum intersection intuicija	12
8.2 Konflikt u bankarskom primeru	13
9 Liveness: kako sistem ipak napreduje?	13
9.1 Pacemaker	13
10 Zasto HotStuff ima tri faze?	14
11 Kompleksnost i responsiveness	14
12 Chained HotStuff	15
12.1 One-chain, two-chain, three-chain	15

13 Plan implementacije simulatora	15
13.1 Predlog jezika: Go	15
13.2 Moduli simulatora	16
13.3 Minimalne strukture podataka	16
13.4 Demo scenariji	17
14 Kratak rečnik	17
15 Pitanja za proveru znanja	18
16 Zaključak	18

1 Motivacija: zasto nam treba konsenzus?

U distribuiranom sistemu nemamo jednu masinu koja sama donosi odluke. Umesto toga imamo vise replika koje treba da se ponasaju kao jedna pouzdana servisna celina. Ako sve ispravne replike izvrse iste komande istim redosledom, i ako su replike deterministicke, one ce završiti u istom stanju. To je osnovna ideja *State Machine Replication* (SMR).

HotStuff resava upravo ovaj problem u Byzantine okruženju. U radu se navodi da BFT SMR protokol treba da obezbedi da neispravne, tj. Byzantine replike ne mogu spreciti ispravne replike da se dogovore o istom redosledu klijentskih komandi, pa zato ispravne replike izvorsavaju komande identicno [1].

Bankarski primer

Zamislamo da cetiri institucije odrzavaju zajednicki ledger:

- B_1 - banka koja trenutno vodi rundu kao lider,
- B_2 - druga banka,
- B_3 - regulator ili audit cvor,
- B_4 - banka koja moze biti Byzantine.

Sistem prima zahteve kao sto su:

- `TRANSFER(Marko, Ana, 500)`,
- `BLOCK_ACCOUNT(ACC-742)`,
- `APPROVE_LOAN(Ana, 10000)`.

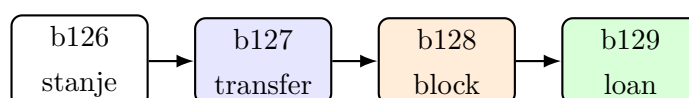
Konsenzus ne odlucuje samo da li je zahtev “validan”, vec na kojoj tacnoj poziciji u zajednickom logu taj zahtev ulazi.

1.1 Zasto je redosled bitan?

U bankarskom sistemu redosled zahteva menja rezultat. Na primer:

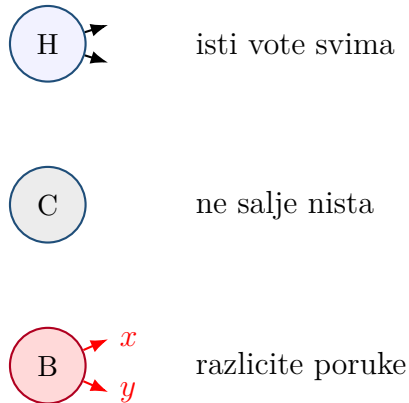
Redosled 1	Redosled 2
1. <code>BLOCK_ACCOUNT(ACC-742)</code>	1. <code>TRANSFER(ACC-742, Ana, 500)</code>
2. <code>TRANSFER(ACC-742, Ana, 500)</code>	2. <code>BLOCK_ACCOUNT(ACC-742)</code>
Transakcija se odbija, jer je racun vec blokiran.	Transakcija moze proci pre blokade.

Zato replike moraju da se dogovore o istom logu, a ne samo o skupu komandi.



2 Byzantine model: sta znaci da replika moze biti zlonamerna?

Crash fault je relativno jednostavan: cvor prestane da radi i ne salje vise poruke. Byzantine fault je tezi: cvor moze slati pogresne, zakasnele ili konfliktne poruke razlicitim replikama.



Byzantine replika ne mora izgledati stalno zlonamerno

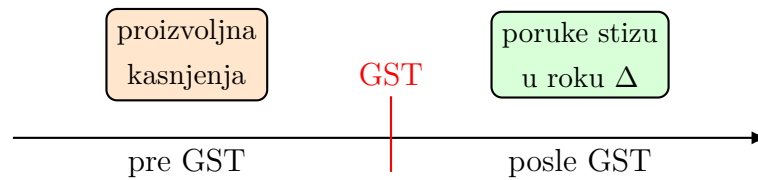
Ona moze neko vreme raditi korektno, zatim u kriticom trenutku poslati konfliktne predloge. Zato protokol ne sme zavisiti od poverenja u jednog lidera ili jedan cvor. Umesto toga, koristi se kombinacija quorum-a, potpisa, QC-a i pravila glasanja.

3 Model HotStuff-a

HotStuff pretpostavlja fiksni skup replika. U radu je model definisan kao sistem sa $n = 3f + 1$ replika, od kojih najvise f moze biti Byzantine. Komunikacija izmedju korektnih replika je point-to-point, autentifikovana i pouzdana. Protokol radi u modelu parcijalne sinhronosti: postoji poznata granica kasnjenja Δ nakon nepoznatog trenutka GST, ali pre GST-a kasnjenja mogu biti proizvoljna [1].

Osnovne pretpostavke

- $n = 3f + 1$: ukupan broj replika.
- f : maksimalan broj Byzantine replika.
- Quorum je $n - f = 2f + 1$ replika.
- Safety mora vaziti uvek, cak i pre GST-a.
- Liveness se garantuje tek kada mreza dovoljno dugo postane stabilna posle GST-a.



Safety uvek; liveness tek kada mreza postane stabilna.

3.1 Zasto bas $n = 3f + 1$?

Za Byzantine toleranciju potrebno je da quorum-i imaju presek u barem jednoj korektnoj replici. Ako je $n = 3f + 1$, quorum ima $2f + 1$ glasova. Dva takva quorum-a zajedno imaju $4f + 2$ mesta, ali ukupno postoji samo $3f + 1$ replika. Zato se dva quorum-a presecaju u najmanje:

$$(2f + 1) + (2f + 1) - (3f + 1) = f + 1$$

replika. Posto najvise f replika moze biti Byzantine, u preseku postoji bar jedna korektna replika. Ta korektna replika nece potpisati dve konfliktne odluke u istoj fazi i istom view-u.

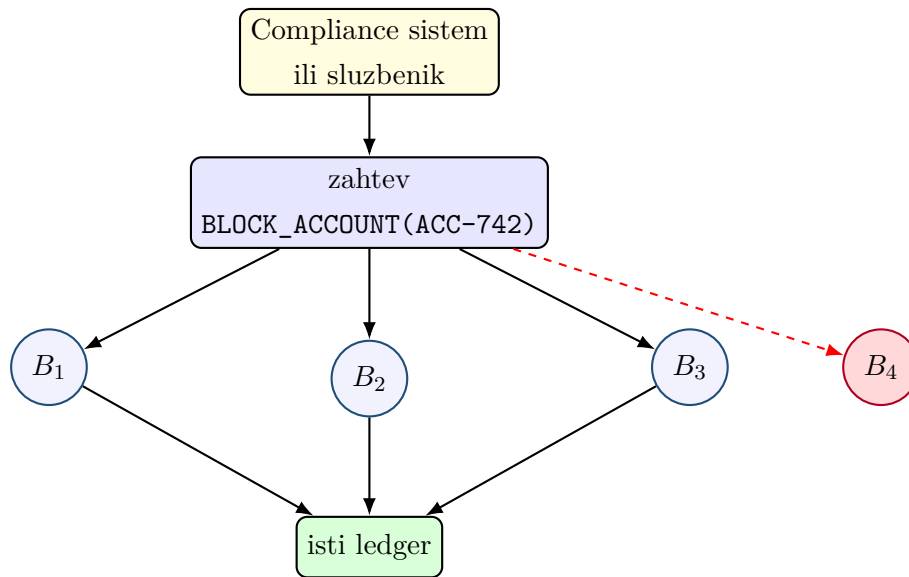
Najmanji primer

Ako je $f = 1$, onda je $n = 4$, a quorum je 3. Dva skupa od po 3 replike u sistemu od 4 replike moraju se preseći u 2 replike. Posto samo jedna moze biti Byzantine, bar jedna u preseku je korektna.

4 Bankarski case study

U nastavku cemo koristiti isti scenario:

- B_1, B_2, B_3 su korektne replike.
- B_4 moze biti Byzantine.
- $n = 4, f = 1, \text{quorum} = 3$.
- Zahtev koji treba odluciti je `BLOCK_ACCOUNT(ACC-742)`.



Sta HotStuff zapravo odlucuje?

HotStuff ne odlucuje samo recenicu: “racun ACC-742 treba blokirati”. Preciznije, odlucuje da node koji sadrzi `BLOCK_ACCOUNT(ACC-742)` ulazi kao sledeci element u zajednicki log, na primer kao `b128`, i da je njegov parent `b127`.

5 Osnovni pojmovi

5.1 Node

Node je jedinica koju lider predlaze. U bankarskom sistemu node sadrzi jednu komandu ili batch komandi, parent link i metapodatke potrebne za protokol.

Listing 1: Pojednostavljena struktura node-a

```

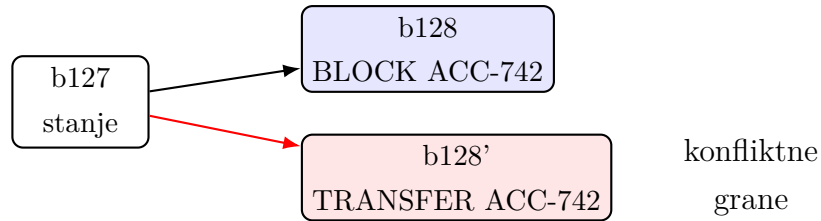
Node {
  id:      b128
  parent:  b127
  command: BLOCK_ACCOUNT(ACC-742)
  view:    12
}
  
```

5.2 Branch

Branch je put od nekog node-a unazad do genesis node-a. Ako node `b128` ima parent `b127`, a `b127` ima parent `b126`, branch je:

$$\text{genesis} \rightarrow \dots \rightarrow b126 \rightarrow b127 \rightarrow b128$$

Dva node-a su konfliktna ako nijedan nije nastavak drugog. Na primer, ako oba b_{128} i b_{128}' pokušavaju da budu dete istog b_{127} , oni su konfliktni.



5.3 Vote

Vote je glas jedne replike za određeni node, u određenoj fazi i određenom view-u. Glas mora biti vezan za tačnu trojku:

$\langle \text{phase, view, node} \rangle$

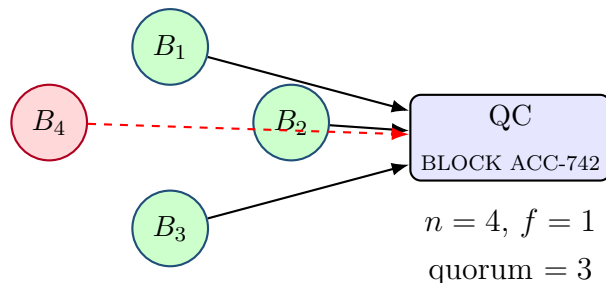
Na primer:

```

Vote {
  replica: B2
  phase:   prepare
  view:    12
  node:    b128
}
  
```

5.4 Quorum Certificate

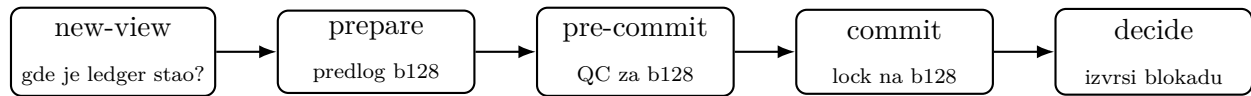
Quorum Certificate, skraćeno QC, je dokaz da je quorum replika glasao za isti node u istoj fazi i istom view-u. U radu je QC definisan kao kolekcija od $n - f$ glasova nad predlogom lidera, a prag je $2f + 1$ [1].



6 Basic HotStuff: faze jednog view-a

Basic HotStuff radi kroz rastuce view-ove. Svaki view ima jednog lidera. Lider predlaže node i pokušava da prikupi glasove u tri glavne faze: prepare, pre-commit i commit. Kada

sakupi commit glasove, salje decide poruku. U radu se naglasava da se branch commit-uje kada lider sakupi quorum glasova kroz tri faze [1].



6.1 New-view: gde je ledger stao?

Kada replika predje u novi view, salje novom lideru poruku **new-view**. Ta poruka nosi najvisi **prepareQC** koji replika zna. Lider ceka $n - f$ takvih poruka i bira najvisi QC, oznacen kao **highQC**.

View 12: lider je B_1

Replike salju:

- $B_2 \rightarrow B_1$: moj najvisi QC je za $b127$,
- $B_3 \rightarrow B_1$: moj najvisi QC je za $b127$,
- $B_4 \rightarrow B_1$: moze poslati tacnu, pogresnu ili nikakvu poruku.

Ako B_1 sakupi tri validne **new-view** poruke i najvisi QC pokazuje na $b127$, novi predlog mora nastaviti od $b127$.

Zasto je highQC bitan?

Novi lider ne sme nasumicno izabrati branch koji mu odgovara. On mora nastaviti najbezbedniju poznatu granu. To je jedan od razloga zasto HotStuff ima jednostavniji view-change od PBFT porodice: lider bira highest QC koji zna, umesto da prenosi komplikovan dokaz o stanju celog sistema.

6.2 Prepare faza: lider predlaze novi zahtev

Lider sada pravi novi node:

```

b128 = Node {
  parent:  b127,
  command: BLOCK_ACCOUNT(ACC-742),
  view:    12
}
  
```

i salje:

```

B1 -> all: PREPARE(b128, justify = QC(b127))
  
```

Svaka replika proverava:

1. da li je predlog iz trenutnog view-a i od pravog lidera,

2. da li *b128* zaista nastavlja node iz opravdanja, tj. `justify.node`,
3. da li vazi `safeNode` pravilo.

Ako provere prodju, replika salje prepare vote lideru.

6.3 Pre-commit faza: lider dokazuje da je prikupio prepare glasove

Kada lider skupi 3 prepare glasa za *b128*, pravi:

`prepareQC(b128)`

Zatim emituje:

```
B1 -> all: PRE_COMMIT(prepareQC(b128))
```

Replike sada vide da je quorum prihvatio isti predlog u prepare fazi. Ako je dokaz validan, salju pre-commit vote.

6.4 Commit faza: replike se zakljucavaju

Kada lider skupi 3 pre-commit glasa, pravi:

`precommitQC(b128)`

Zatim salje:

```
B1 -> all: COMMIT(precommitQC(b128))
```

Ovo je trenutak kada replike postavljaju:

```
lockedQC = precommitQC(b128)
```

Sta znaci lock?

Lock je lokalno obećanje replike: “necu glasati za konfliktan branch koji ne nastavlja moj locked node, osim ako mi novi predlog ne pokaze noviji QC koji opravdava promenu”. Lock ne znaci da je komanda vec izvršena, nego da replika cuva safety za buducu odluku.

6.5 Decide faza: zahtev se izvrsava

Kada lider sakupi 3 commit glasa, pravi:

`commitQC(b128)`

Zatim salje:

```
B1 -> all: DECIDE(commitQC(b128))
```

Kada korektna replika primi validan commitQC, ona izvršava komandu iz branch-a:

```
execute BLOCK_ACCOUNT(ACC-742)
```

Sada sve korektne replike imaju isti rezultat: racun ACC-742 je blokiran na istoj poziciji u ledger-u.

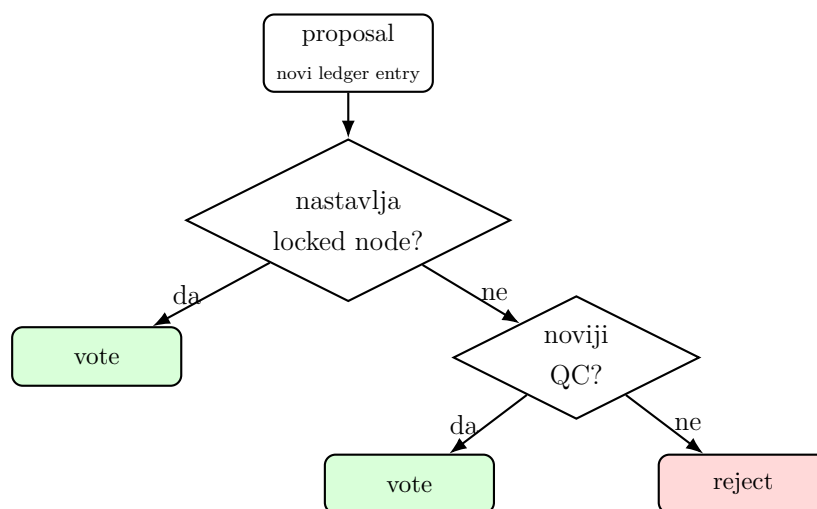
7 safeNode pravilo

safeNode je srce HotStuff safety/liveness ravnoteze. U pseudokodu iz rada, replika prihvata node ako vazi bar jedno od dva pravila [1]:

node extends from lockedQC.node

ili

qc.viewNumber > lockedQC.viewNumber



7.1 Safety rule

Ako novi predlog nastavlja branch na koji je replika zaključana, onda glasanje ne može ugroziti prethodno zaključanu istoriju.

Bezbedan nastavak

Ako je replika zaključana na:

$$b_{127} \rightarrow b_{128} = \text{BLOCK_ACCOUNT}(\text{ACC-742})$$

onda sme glasati za:

$$b_{129} = \text{APPROVE_LOAN}(\text{Ana}, 10000)$$

ako je b_{129} dete od b_{128} . To ne menja cinjenicu da je blokada ranije u ledger-u.

7.2 Liveness rule

Ako predlog ne nastavlja locked node, replika ga ipak može prihvatiti ako predlog nosi QC iz novijeg view-a od njenog lock-a. To omogućava sistemu da se oporavi od zastarelih lock-ova i loših lidera.

Zasto ne sme samo safety rule?

Ako bi replike uvek odbijale sve što ne nastavlja njihov lock, sistem bi mogao ostati zaglavljen. Liveness rule dozvoljava napredak, ali samo kada postoji noviji QC kao dokaz da je quorum već presao na noviju fazu istorije.

8 Safety: zasto dve konfliktne odluke ne mogu proci?

Safety u HotStuff-u znaci: dve korektne replike ne mogu commit-ovati konfliktne node-ove. U bankarskom primeru, ne sme se dogoditi da jedna ispravna replika izvrši:

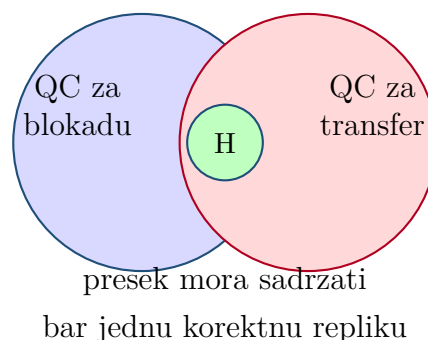
BLOCK_ACCOUNT(ACC-742)

a druga ispravna replika, na istoj poziciji ledger-a, izvrši konfliktan zahtev:

TRANSFER(ACC-742, Ana, 500)

8.1 Quorum intersection intuicija

Svaki commit zahteva QC od $2f + 1$ glasova. Dva takva QC-a se moraju preseći u bar jednoj korektnoj replici. Korektna replika ne glasa za konfliktan branch ako joj safeNode to ne dozvoljava.



Najkrace objasnjenje safety-ja

Byzantine replika može da laže, ali ne može sama napraviti QC. Da bi nastala dva konfliktna commitQC-a, neka korektna replika bi morala da pomogne oba konflikta. Lock i safeNode pravila su dizajnirani tako da se to ne desi.

8.2 Konflikt u bankarskom primeru

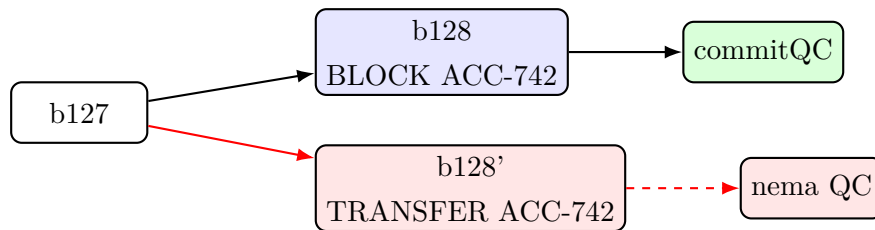
Pretpostavimo da je B_4 Byzantine i pokušava da progura:

$$b_{128}' = \text{TRANSFER}(\text{ACC-742}, \text{Ana}, 500)$$

umesto:

$$b_{128} = \text{BLOCK_ACCOUNT}(\text{ACC-742})$$

Dva node-a su sibling-i, jer oba pokušavaju da budu dete od b_{127} .

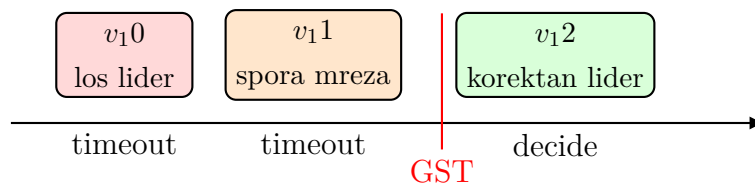


Ako su korektne replike već zaključane na b_{128} , one neće glasati za b_{128}' jer b_{128}' ne nastavlja zaključanu granu i nema noviji QC koji bi opravdao promenu.

9 Liveness: kako sistem ipak napreduje?

Safety sama po sebi nije dovoljna. Sistem koji nikad ne donosi odluke je bezbedan, ali beskoristan. Liveness znači da će sistem, pod pravim uslovima, na kraju doneti odluku.

HotStuff razdvaja safety logiku od liveness logike. Safety je u pravilima glasanja, QC-u, lock-u i safeNode-u. Liveness je u Pacemaker-u, tj. mehanizmu koji pomaze replikama da predju u novi view i da se eventualno okupe oko korektnog lidera.



9.1 Pacemaker

Pacemaker nije zadužen da proverava da li je predlog bezbedan. On pomaze da replike budu u istom view-u dovoljno dugo i da se lider rotira dok se ne pojavi korektan lider. U radu je liveness mehanizam izdvojen u Pacemaker, čime se safety mehanizam čisto odvaja od liveness mehanizma [1].

Jednostavna verzija Pacemaker-a može koristiti:

- rotirajući raspored lidera,

- timeout za svaki view,
- eksponencijalni backoff: ako nema odluke, sledeci timeout je duzi.

Listing 2: Intuicija Pacemaker-a

```
on timeout(view):
    view = view + 1
    timeout = 2 * timeout
    send NEW_VIEW(highestQC) to leader(view)
```

10 Zasto HotStuff ima tri faze?

Na prvi pogled deluje da su tri faze nepotrebne: ako imamo prepareQC, zasto ne commit odmah? Problem je view-change. U dve faze moze nastati situacija u kojoj jedna korektna replika ima najvisi QC i lock, ali novi lider ne zna za taj QC jer ga nije dobio u svojim $n - f$ poruka. Tada lider moze predloziti konfliktan branch i sistem se moze vrteti bez odluke.

Rad eksplicitno objasjava da HotStuff dodaje jos jednu fazu kao malu cenu u latenciji, ali time dobija jednostavniji leader replacement i optimistic responsiveness [1].

Tri faze kao cena za jednostavniji view-change

Dodatna faza omogucava da se replike “predomisle” samo uz pravilan dokaz, bez toga da novi lider mora nositi komplikovan PBFT-style leader proof. Zato novi lider moze jednostavno uzeti highest QC koji zna.

11 Kompleksnost i responsiveness

HotStuff je vazan jer kombinuje dve osobine:

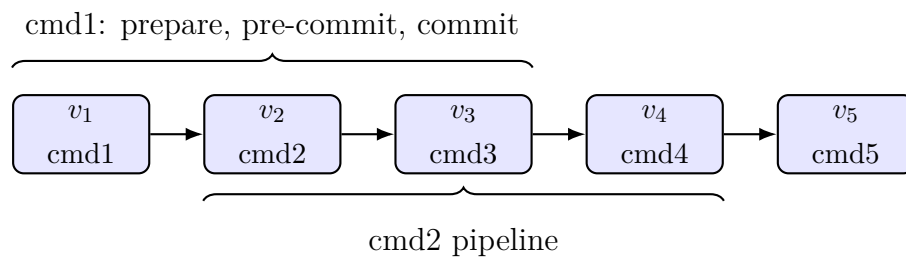
- **Linear View Change:** posle GST-a korektan lider salje samo $O(n)$ authenticator-a da dovede sistem do odluke, cak i kada se lider menja.
- **Optimistic Responsiveness:** posle GST-a korektan lider ne mora da ceka unapred poznatu maksimalnu granicu Δ , vec napreduje tempom stvarnog kasnjenja poruka.

U tabeli performansi u radu, HotStuff ima $O(n)$ authenticator complexity za korektnog lidera, $O(n)$ za leader failure/view-change i $O(fn)$ za f neuspelih lidera, uz responsiveness [1].

Protokol	Korektan lider	View-change	Responsiveness
PBFT	$O(n^2)$	$O(n^3)$	da
SBFT	$O(n)$	$O(n^2)$	da
Tendermint/Casper	$O(n^2)$	$O(n^2)$	ne
HotStuff	$O(n)$	$O(n)$	da

12 Chained HotStuff

Basic HotStuff je dobar za razumevanje, ali ima faze koje lice jedna na drugu. Chained HotStuff pojednostavljuje protokol tako sto svaki novi proposal u novom view-u istovremeno služi kao sledeća faza za prethodne predloge. U radu se to opisuje kao pipeline: prepare za view $v + 1$ istovremeno služi kao pre-commit za view v , a prepare za view $v + 2$ služi kao commit za view v [1].



12.1 One-chain, two-chain, three-chain

U Chained HotStuff-u se umesto eksplicitnih faza posmatra oblik lanca:

- **One-chain:** node ima QC koji pokazuje na direktnog parent-a.
- **Two-chain:** postoje dva uzastopna takva koraka; to odgovara pre-commit logici.
- **Three-chain:** postoje tri uzastopna koraka; to odgovara commit-u starijeg node-a.

Kako pamtiti Chained HotStuff?

Basic HotStuff: jedna komanda prolazi kroz više faza u jednom view-u. Chained HotStuff: svaki novi view nosi novi predlog, a niz predloga zajedno predstavlja faze za starije komande. Time protokol postaje pipeline-friendly.

13 Plan implementacije simulatora

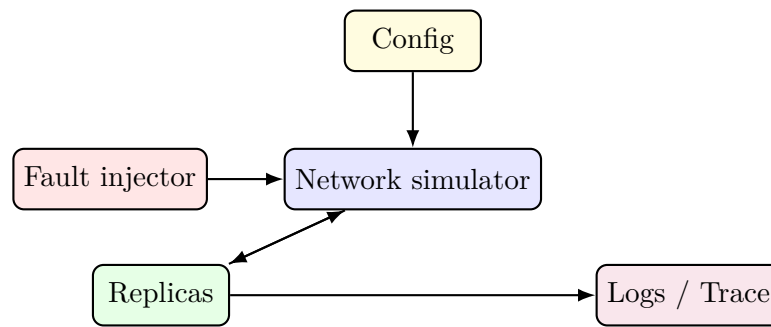
Za projekat je dovoljno napraviti simulator koji jasno pokazuje faze, glasove, QC-eve, lock i view-change. Cilj nije produkcionni blockchain, nego razumljiva demonstracija safety-ja i liveness-ja.

13.1 Predlog jezika: Go

Go dobro odgovara ovom projektu:

- goroutine može predstavljati repliku,
- channel može predstavljati tok poruka,
- context i select mogu modelovati timeout i view-change,
- statički tipovi olaksavaju jasne strukture za Message, Vote, QC i Node.

13.2 Moduli simulatora



13.3 Minimalne strukture podataka

Listing 3: Minimalne strukture za simulator

```

type Command struct {
    Kind    string // TRANSFER, BLOCK_ACCOUNT, APPROVE_LOAN
    Args    []string
}

type Node struct {
    ID       string
    Parent   string
    Command  Command
    View     int
}

type Vote struct {
    Replica  string
    Phase    string
    View     int
    NodeID   string
}

type QC struct {
    Phase  string
    View   int
    Node   string
    Votes  []Vote
}

type ReplicaState struct {
    View     int
    Tree     map[string]Node
  
```



```

    LockedQC  QC
    PrepareQC QC
    Ledger    [] Command
}

```

13.4 Demo scenariji

Za odbranu projekta korisno je imati tri scenarija:

1. **Happy path:** B_1 je korektan lider i uspesno commit-uje `BLOCK_ACCOUNT(ACC-742)`.
2. **Byzantine lider:** B_4 salje konfliktne predloge, na primer blokadu jednoj grupi, a transfer drugoj grupi. Sistem ne sme commit-ovati oba.
3. **Pad lidera:** lider sakupi neke glasove, ali ne završi decide. Timeout vodi u novi view, novi lider skuplja `new-view` poruke i nastavlja od highest QC.

14 Kratak recnik

Pojam	Znacenje
BFT	Byzantine Fault Tolerance; otpornost na proizvoljno zlonamerno ponasanje replika.
SMR	State Machine Replication; sve replike izvorsavaju iste komande istim redosledom.
View	Runda protokola sa jednim liderom.
Leader	Replika koja u datom view-u predlaze novi node i skuplja glasove.
Node	Predlog koji sadrzi komandu, parent link i protokolske metapodatke.
Branch	Put od nekog node-a do genesis node-a.
Vote	Potpisani glas replike za fazu, view i node.
QC	Quorum Certificate; dokaz da je quorum glasao za isti tuple.
lockedQC	Lokalni lock replike koji sprecava glasanje za konfliktan branch.
prepareQC	Najvisi poznati prepare dokaz koji se salje u new-view poruci.
safeNode	Pravilo koje odlucuje da li replika sme da glasa za predlog.
GST	Global Stabilization Time; trenutak posle koga poruke izmedju korektnih replika stizu u ogranicenom vremenu.
Pacemaker	Liveness modul koji pomaze replikama da se sinhronizuju u view-u i rotira lidere.

15 Pitanja za proveru znanja

1. Zasto HotStuff ne odlucuje samo o validnosti transakcije, nego o njenoj poziciji u logu?
2. Sta Byzantine replika moze da uradi sto crash replika ne moze?
3. Zasto je za $f = 1$ potrebno ukupno $n = 4$ replike?
4. Sta tacno dokazuje QC?
5. Koja je razlika izmedju `prepareQC`, `precommitQC` i `commitQC`?
6. Kada replika postavlja `lockedQC`?
7. Zasto lock ne znaci isto sto i execute?
8. Objasni `safeNode` pravilo kroz primer blokade racuna.
9. Zasto dva konfliktna node-a ne mogu oba biti commit-ovana?
10. Zasto HotStuff uvodi trecu fazu?
11. Sta radi Pacemaker, a sta ne radi?
12. Kako Chained HotStuff koristi pipeline?

16 Zakljucak

HotStuff je BFT konsenzus protokol koji resava problem dogovora o redosledu komandi u replikovanom sistemu. Njegova glavna ideja je da se bezbednost cuva kroz QC, lock i `safeNode` pravila, dok se napredak prepusta Pacemaker-u. U bankarskom case study-ju to znaci da sve korektne banke i regulator vide isti ledger: ako je blokada racuna commit-ovana pre transakcije, sve korektne replike ce tu blokadu izvršiti pre transakcije.

Najvaznije recenice za pamcenje su:

Tri recenice za pamcenje

- QC je dokaz da je quorum glasao za isti predlog.
- Lock je lokalno obećanje replike da neće podržati konfliktan branch bez novijeg QC-a.
- Commit je globalno bezbedna odluka koja može da se izvrši u svim korektnim replikama.

References

- [1] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. *HotStuff: BFT Consensus with Linearity and Responsiveness*. Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC 2019), 2019.